

Phase 3: Bounded-Log Tree CRDT with RAFT-Driven Log Compaction

CS 403/534 - Distributed Systems
Course Project, Spring 2025-2026

May 24, 2026

Deadline: June 3, 23:55 (Turkish Time)

Abstract

Phase 3 closes out the Tree CRDT project by replacing the causal-stability-driven log compaction of Phase 2 with an agreement-based compaction protocol built on top of RAFT. Along the way, the data structure reverts from a multi-value DAG back to a single-valued tree: concurrent **Move** and **Delete** operations are now totally ordered by a deterministic ranking on vector timestamps, rather than coexisting as parallel versions, and the Phase 2 Move-Wins policy is retired. Each replica maintains a *checkpoint vector* that tracks how far its log has been flattened into a *stable tree*, advances a local *safe index* on its log according to a set of admissibility checks, and then participates in a RAFT instance whose committed prefix governs when entries can be physically erased. This document is written as a *delta* on Phases 1 and 2 and should be read alongside them.

1 Introduction

Phase 2 introduced vector clocks, the **Delete** operation, multi-value tree state, and a first cut at log compaction driven by causal stability. The compaction routine of Phase 2 was *purely local*: each replica computed a causal-stability threshold τ from its per-peer progress map and discarded any log entry whose timestamp fell strictly below τ . That mechanism is sound when peer-progress data are exchanged promptly and reliably, but it has two limitations that Phase 3 addresses head-on.

First, two replicas can compact different prefixes of their logs at different times, and there is no protocol-level guarantee that an entry compacted by one replica is also known to all others. Under the Phase 2 wire assumptions, this is fine — the entry would only have been compacted if it were causally stable everywhere — but as soon as we want to make compaction *coordinated* (so that the log on every replica looks the same up to the compaction frontier), we need an agreement protocol. Second, the partial order on vector timestamps leaves room for genuinely concurrent operations on the same child, which Phase 2 resolved by maintaining a multi-value set per child and a Move-Wins tiebreaker.

Phase 3 trades that permissiveness for a stricter, totally ordered view of operations, which collapses the multi-value tree back into a single-valued one and removes the need for Move-Wins entirely.

Conceptually, Phase 3 keeps the vector clock as a causality tracker but layers a deterministic *total order* on top of it. The order is a lexicographic refinement: first by the sum of the vector entries, then by left-to-right fieldwise comparison as a tiebreaker for vectors with the same sum. This ordering respects happens-before — if $V \leq W$ under the partial order of Phase 2, then V comes no later than W under the total order of Phase 3 — and it is the only place in the project where two concurrent operations are explicitly forced apart.

The second conceptual change is checkpointing. Phase 2 measured compactability with a single threshold τ computed from per-peer progress. Phase 3 measures it with a *checkpoint vector* C , one entry per replica, recording how many operations from each source the local stable tree has absorbed. The replica advances C field by field as the entries at the front of its log satisfy a set of admissibility checks (Section 2.6). The pointer into the log at which advancement is being attempted is called the *safe index*; everything below the safe index is, by local information alone, ready to be folded into the stable tree.

Local readiness is not sufficient, however, to physically erase log entries: a replica must also know that every other replica has caught up. To obtain that guarantee, Phase 3 lays a RAFT instance over the operation logs. RAFT’s role is restricted: it does not attempt to commit the whole log, but only the prefix up to the leader’s current safe index, and each replica is allowed to delete log entries below its RAFT commit index.

The reference RAFT implementation lives in a new sub-package *src/tree_crdt/raft*; it communicates using gRPC over a dedicated **RAFT_BASE** port range and runs on a daemon thread embedded in each **Replica**. Section 2.9 describes how the **Replica** drives that thread, the shape of the commands it proposes to RAFT, the wire format used between replicas, and the two-stage shutdown that drains it cleanly at the end of a run.

A third change is the project’s termination story. Phase 2’s checkpointing already had a subtle deadlock failure mode: if a replica stops generating operations, its field in everyone else’s checkpoint vector ceases to advance, and admissibility checks on the right of that field can block indefinitely. Phase 3 makes the solution explicit: the configuration file declares, for each replica, the total number of operations it will ever issue, and every replica is aware of these numbers. The checkpointing routine treats a saturated field as terminally saturated and does not look for higher values from that source.

The remainder of this document specifies these changes section by section. As in Phase 2, the spec is given at the level of method contracts and required semantics; the algorithmic and data-structural choices inside those contracts are yours to make.

2 Description

2.1 Total Ordering of Vector Timestamps

Each replica continues to maintain a vector clock like the one in Phase 2 (Section 2.1 of the Phase 2 specification), but with a modified update rule: the

update rule on a local event is $V_i[i] += 1$, and on receipt of a remote operation tagged with vector W the rule is the component-wise maximum. The `VectorClock` class and its four comparison helpers (`timestamp_le`, `timestamp_lt`, `timestamp_eq`, `timestamp_concurrent`) carry over to Phase 3 with a modification on the ordering of timestamps.

Precisely, what is new in Phase 3 is a *total order* layered on top of the partial order. Given two vector timestamps t_1 and t_2 over the same key set, define

$$\text{sum}(t) = \sum_k t[k].$$

The total order $\prec$ is then:

- If $\text{sum}(t_1) < \text{sum}(t_2)$, then $t_1 \prec t_2$.
- If $\text{sum}(t_1) = \text{sum}(t_2)$, compare t_1 and t_2 fieldwise from left to right (i.e. from $k = 0$ upward). At the first index k where the two vectors differ, the vector with the smaller value at k is the smaller of the two under $\prec$.

Causality preservation. This order is a refinement of the Phase 2 partial order: if $V \leq W$ under the partial order, then $V \preceq W$ under $\prec$. This is because $V \leq W$ implies $V[k] \leq W[k]$ for every k , so the sum of V is no greater than the sum of W ; and on equality of sums, $V \leq W$ forces $V = W$, so the tiebreaker is moot. Concurrent timestamps are the only case in which $\prec$ adds new information: under $\prec$ they are forced into one order or the other rather than coexisting as peers.

Why the tiebreaker? Two operations with the same vector sum can either be equal (impossible for two distinct operations under our update rules, but worth ruling out explicitly) or concurrent. The left-to-right fieldwise tiebreaker is a convention — any deterministic rule that agrees across replicas would do — and you must not invent your own. It is also the rule that drives the admissibility checks of the safe-index routine (Section 2.6), so deviating from it will cause subtle bugs there.

2.2 Reverting to a Single-Valued Tree

A consequence of the total order $\prec$ is that the multi-value representation introduced in Phase 2 (Section 2.4 of the Phase 2 specification) is no longer necessary. When two operations target the same child concurrently, the total order picks one as smaller and the other as larger; the smaller one is treated as preceding the larger one for all purposes, exactly as if it had causally happened before it. The per-child set of versions therefore degenerates to a single version at all times.

The data structure is consequently a tree again, as in Phase 1, rather than the DAG as in the version of Phase 2.

Note on backward and forward compatibility. Backward and forward compatibility with Phases 1 and 2 are not a concern in Phase 3.

2.3 Delete in Phase 3

The **Delete** operation continues to exist as in Phase 2; what changes, however, is the resolution policy: there is no longer a Move-Wins preference. A concurrent **Move/Delete** pair on the same child is ordered by $\prec$ and the larger one (under $\prec$) wins unconditionally. Depending on the vector sums and the tiebreaker, a deletion may win against a concurrent move or lose to it; you must not bias either way.

2.4 Two Trees, One Replica: Current and Stable

Each replica maintains *two* **Tree** instances:

- The *current tree* is the tree obtained by applying all operations visible to this replica, including those still sitting in the log above the checkpoint vector. This is the tree the user reads when querying the replica's state, and it is what the undo-do-redo sequence operates on.
- The *stable tree* is the tree obtained by applying only those operations that the replica has *checkpointed* — i.e. folded into the stable tree and removed from the front of the log. The stable tree corresponds to the prefix of the log that the replica's checkpoint vector (Section 2.5) accounts for.

The stable tree of Phase 3 plays the same role as the *snapshot* of Phase 2 (Section 5.7.3 of the Phase 2 specification): it is the frozen part of the tree state, provably immune to retroactive change under the local admissibility checks.

2.5 The Checkpoint Vector and the Safe Index

The replica's progress toward log compaction is summarised by two pieces of state on top of the two trees of Section 2.4: the *checkpoint vector* and the *safe index*.

Checkpoint vector. The checkpoint vector C has the same set of keys as the local vector clock. The entry $C[k]$ records the number of *messages* (operations) originating from replica k that the local stable tree has absorbed — not, for instance, the latest clock value seen from replica k . When a log entry whose source is replica k is folded into the stable tree, $C[k]$ is incremented by one and the entry is removed from the front of the log.

Safe index. The safe index is a pointer to the operation log; it marks the boundary at which the replica has stopped advancing the checkpoint vector. Everything strictly to the left of the safe index has already been checkpointed and erased; the entry at the safe index is the next candidate for checkpointing, if it passes the admissibility checks of Section 2.6.

The safe index is monotonically non-decreasing: once an entry has been checkpointed, the safe index moves past it and never returns.

2.6 Advancing the Safe Index: Admissibility Checks

The safe-index advancement routine is the local half of Phase 3’s log compaction — the half that does not require any agreement with peers. It runs whenever the front of the log changes, and it attempts to fold as many entries as possible into the stable tree before stopping.

Let the candidate log entry at the safe index have timestamp t and source replica k . The four admissibility checks are:

1. *Single-field exceedance.* The vector t exceeds the checkpoint vector C in exactly one field. That is, there is exactly one index j at which $t[j] > C[j]$, and at every other index $\ell \neq j$, $t[\ell] = C[\ell]$.
2. *Source-field match.* The field at which t exceeds C is the field corresponding to the source replica of the entry: $j = k$.
3. *Unit increment.* The difference in that field is exactly one: $t[k] = C[k] + 1$.
4. *Tiebreak coverage.* For every field index $\ell > k$ (i.e. every field to the right of k in the left-to-right fieldwise comparison), there exists *some* entry in the log — not necessarily the one at the safe index — whose timestamp t' satisfies $t'[\ell] = C[\ell] + 1$. (See the explanation below for why this check exists.)

If all four checks pass, the safe index is incremented: the entry is applied to the stable tree, $C[k]$ is incremented by one, and the entry is removed from the front of the log. The routine then retries with the new front-of-log entry. When any check fails, the routine stops and leaves the rest of the log alone.

Why each check exists. The first three checks in accumulation enforce that the candidate is exactly the “next” operation to be checkpointed from its source, relative to the current checkpoint vector. The first check forbids a candidate whose timestamp is ahead of C on more than one field (e.g. $C = \langle 3, 5, 8 \rangle$ and $t = \langle 4, 5, 9 \rangle$ from replica C): if such a candidate were admitted, the missing prior operation from replica A could still arrive and would have to be inserted before t under the total order $\prec$, which would in turn force an undo of t from the stable tree — a contradiction with the stable tree’s role.

The second and third checks enforce that the single exceeded field is the source’s own field, and that the exceedance is by one. Together, they ensure that the candidate is the very next operation generated by its source after the most recent one already checkpointed from that source.

The fourth check is the subtlest. Imagine $C = \langle 3, 5, 8 \rangle$ on a three-replica system with sources A, B, C , and let the candidate at the safe index be a $\langle 3, 6, 7 \rangle$ entry from B . The first three checks pass: the candidate exceeds C only at field B , by exactly one. But the sum of the candidate is 16, the same as the sum of $\langle 3, 5, 9 \rangle$ from C , and the tiebreaker under $\prec$ would place $\langle 3, 5, 9 \rangle$ *before* $\langle 3, 6, 7 \rangle$. If some operation from C with timestamp $\langle 3, 5, 9 \rangle$ has not yet arrived at this replica, checkpointing the B entry would forbid inserting the C entry into the stable tree in the correct position, since the stable tree is no longer revisable.

The fourth check rules this out: it requires that, for every field $\ell > k$ in the left-to-right tiebreaker, some entry already *visible* in the log advances field

ℓ past $C[\ell]$. The intuition is that if such an entry is visible, then any concurrent operation from a source ℓ that could rank before the candidate under $\prec$ has either already arrived (and is the witness) or, by the partial-order constraints on vector clocks, cannot still be in flight without violating causality with the witness.

When the routine runs. The safe-index routine is only triggered when a new *minimum-entry* candidate appears at the safe index — concretely, when a new operation is inserted into the log at a position at or before the safe index, or when a previous successful advancement removed the front entry and the routine is iterating forward. Operations inserted strictly after the safe index do not trigger the routine: they cannot become the candidate without prior advancements removing intermediate entries first.

2.7 Termination and Saturated Fields

Like the total-order broadcast algorithms it descends from, the safe-index routine of Section 2.6 can deadlock if a replica stops generating operations. Concretely: once replica k issues its last operation, no further entry with $t[k] > C[k]$ can ever arrive, and any admissibility check that depends on a future advance of field k blocks forever. The fourth check, in particular, will block whenever the candidate has a higher-indexed source and a right-of-source field of k .

To avoid this, the configuration file, *.env*, declares, for each replica, the total number of operations that replica will ever generate. Every replica is told these numbers at startup and stores them as a vector L , where $L[k]$ is the operation limit for replica k . The safe-index routine consults L as follows:

- In the fourth admissibility check (Section 2.6), the requirement that some log entry advances field ℓ past $C[\ell]$ is *skipped* when $C[\ell] = L[\ell]$. That is: if replica ℓ has already had all of its operations checkpointed (and there will therefore never be a future entry with $t[\ell] > C[\ell]$), the check is treated as trivially satisfied.

The other three checks are unaffected; they concern the candidate itself and do not depend on prospective future operations.

2.8 Insertion Into the Log and the Undo-Do-Redo Sequence

When a new operation arrives at a replica, it must first be inserted into the correct location in the log according to its timestamp under the total order $\prec$ of Section 2.1. If insertion would land at the end of the log, the operation is appended and applied to the current tree directly. Otherwise — i.e. if the operation has to be inserted before some existing log records — the entries strictly after the insertion point must first be undone (in reverse order) on the current tree, the new operation is inserted at the insertion point and applied to the current tree, and the previously undone entries are redone in their original order.

This is the same undo-do-redo sequence as in Phase 1, with two differences: the ordering relation is now the total order $\prec$, and the Move-Wins /

log-insertion-rollback-redo paths of Phase 2 are removed. In particular, the "applied" metadata key introduced in Phase 2 is no longer needed: there is no longer a notion of a past entry having its applied-flag flipped, so the field can be dropped from the metadata schema.

After the current tree is consistent again, the safe-index routine of Section 2.6 should be invoked to fold as many entries as possible into the stable tree.

2.9 Log Compaction with RAFT

The safe-index routine of Section 2.6 establishes, for each replica independently, that a prefix of its log can be folded into its stable tree. What it does not establish is that every *other* replica has reached the same prefix. Phase 3 layers a RAFT instance over the operation logs of the replicas to obtain that guarantee, after which entries can be physically deleted.

The required behaviour is:

- The classical RAFT formulation has a leader that attempts to commit all entries of its log. In Phase 3, the end of the log as RAFT sees it is the leader's current *safe index*, not the leader's most recent operation. The leader does not propose anything past its own safe index.
- In addition to the safe index, the RAFT instance maintains a *commit index* on each replica: the position up to which all replicas (or a sufficient majority, by RAFT's standard rules) have caught up with the leader's proposals. The commit index never overtakes the safe index, and the safe index never overtakes the log end.
- When a replica observes its commit index advance, it is permitted to physically erase all log entries strictly below the new commit index. These entries have, by the time the commit index reaches them, both been folded into the stable tree (a precondition for the leader having proposed them) and been acknowledged by a majority of replicas (which is what RAFT commitment means). Erasure is purely local; no further round of communication is required.

2.9.1 Reference Implementation Overview

The reference RAFT implementation lives under a new sub-package `src/tree_crdt/raft` that is bundled with the Phase 3 starter package. It exposes four classes that the rest of the library interacts with:

- **RaftNode**: the consensus state machine. It owns the current term, role (FOLLOWER, CANDIDATE, or LEADER), the per-peer `next_index` and `match_index` tables, an election timer and a heartbeat timer, and the local **RaftLog**. The class is generic in the command type carried by log entries; the **Replica** instantiates it as **RaftNode[dict]**.
- **RaftLog**: a list of **LogEntry(term, command)** records with a sentinel at relative index 0 and a movable *base index* that lets a prefix be physically discarded via the `compact(index)` method. The base index is what RAFT uses internally to keep its log size bounded once compaction has happened;

`last_index()`, `entries_from(i)`, and `compact(i)` all operate in terms of absolute indices.

- **GrpcRaftServer**: the inbound side of the wire. It binds a gRPC server to a per-replica address `host:RAFT_BASE+id`, and routes incoming `RequestVoteRpc` and `AppendEntriesRpc` calls to `RaftNode.handle_request_vote(...)` and `RaftNode.handle_append_entries(...)`, respectively.
- **GrpcTransport**: the outbound side. It lazily opens an `insecure_channel` to each peer on first use, encodes outgoing `RequestVote` and `AppendEntries` messages with a `CommandCodec` (the default codec JSON-encodes the command payload), and feeds the responses back into the local `RaftNode` via `handle_request_vote_response(...)` and `handle_append_entries_response(...)`.

The wire format is fixed by the Protobuf schema in `src/tree_crdt/raft/raft.proto` and is intentionally close to the canonical RAFT formulation: a `LogEntry` is a (`term`, `command`) pair, with the command serialised as opaque bytes; `RequestVote` and `AppendEntries` carry the standard term, candidate/leader ID, log-tail metadata, and leader-commit fields. The `CommandCodec` is the only place in which the `Replica`'s domain-level command shape is allowed to leak through, and it defaults to JSON; the Protobuf side never inspects the bytes.

The Protobuf schema also includes an `InstallSnapshot` RPC used to transfer a compacted checkpoint when a follower is too far behind. The `Snapshot` message carries (`term`, `leader_id`, `last_included_index`, `last_included_term`, `data`) where `data` encodes the compacted stable-tree snapshot (the `SnapshotPayload` in the proto). In the reference implementation `GrpcTransport` exposes a `send_install_snapshot` path and the `RaftNode.handle_install_snapshot` handler is expected to:

- reject snapshots with stale terms;
- apply the snapshot atomically via the `Replica`'s snapshot-apply callback, restoring the stable tree, checkpoint vector and local `log_offset`;
- advance the RAFT state (`state.commit_index` and `state.last_applied`) to `last_included_index`; and
- compact the local `RaftLog` base index to `last_included_index` so the truncated prefix can be discarded.

The handler must be idempotent (repeated or overlapping snapshots must not corrupt state) and must permit subsequent `AppendEntries`, since a leader may follow a snapshot with new log entries.

2.9.2 Embedding RAFT in the Replica

Each `Replica` owns a single `RaftNode` instance and the two gRPC sides of it. The `Replica` constructor allocates a `RaftNode` (passing `node_id=self.id`, the full list of peer IDs, and an `apply` callback), then constructs and starts the `GrpcRaftServer` on `tcp://host:RAFT_BASE+id`, then constructs the `GrpcTransport` with the dictionary of peer addresses `{peer_id ↦ host[peer_id]:RAFT_BASE+peer_id}`, and finally hands that transport back to the `RaftNode` via `set_transport(...)`.

The chicken-and-egg construction (node before transport, transport before being attached to node) is deliberate: the `Replica` needs the node's address to bind the server, but the transport needs the node to know where to deliver responses.

A dedicated daemon thread, `--run-raft`, drives the consensus state machine. Once per loop iteration it:

1. Calls `RaftNode.tick(elapsed_ms)` with the wall-clock delta since the previous iteration. The `tick` method advances the heartbeat timer (when leader) or the election timer (otherwise) and triggers `start_election()` or `--send_heartbeats()` as needed. `tick` also invokes `--apply_committed()` when a new commit index arrives, which is the path through which the `apply` callback fires.
2. Drains any `(safe_index_value, raft_log_index)` pairs that the gRPC thread has enqueued through the `apply` callback while the consensus state machine was advancing the commit index. The drain takes a short-lived `Lock` that is independent of the `Replica`'s main `RLock`, so the gRPC thread never blocks on user-level CRDT work.
3. Attempts to advance the local safe index via `--try_advance_safe_index` (Section 2.6). If the safe index has advanced past the value most recently proposed to RAFT and this replica is currently the leader, it submits the new value to the `RaftNode` as a client command `{"op": "safe_index", "value": k}` via `client_append(...)` (followers do not propose).
4. Sleeps for ≈ 10 ms before iterating. The `RaftNode` uses an election timeout drawn uniformly from [150, 300] ms and a 50ms heartbeat interval by default.

The only command type that the `Replica` ever proposes is `safe_index`; the RAFT log therefore carries a sparse sequence of compaction targets rather than the full operation stream. When such a command is committed (i.e. replicated to a majority and applied by RAFT's `--apply_committed`), the `Replica`'s `apply` callback enqueues the new value together with the RAFT log index at which it was committed. The draining step in `--run-raft` then forwards the pair to `--apply_safe_index_commit`, which updates `--raft_committed_safe_index` and calls `--compact_log`; that routine drops all operation-log entries strictly below `min(--raft_committed_safe_index, --safe_index_abs)`, raises `--log_offset` accordingly, and then asks the `RaftLog` to truncate itself to the corresponding RAFT log index. The CRDT operation log and the RAFT log advance in lockstep, but neither side outruns its local proof of safety.

InstallSnapshot handling. When a peer sends an `InstallSnapshot` RPC, the `RaftNode` should forward the snapshot bytes to the owning `Replica` rather than applying them inline on the gRPC thread. The `Replica` should apply snapshots from the RAFT daemon thread (for example via the same short-lived lock used to drain committed entries) so the snapshot is applied atomically with respect to other CRDT state changes. Applying a snapshot must:

- reject snapshots with stale terms;

- restore the stable-tree bytes, checkpoint vector and local `log_offset` atomically via the Replica's snapshot-apply callback;
- advance RAFT's `state.commit_index` and `state.last_applied` to `last_included_index`; and
- compact the local `RaftLog` base index to `last_included_index` so the truncated prefix can be discarded.

The snapshot handler must be idempotent (repeated or overlapping snapshots must not corrupt state) and must permit subsequent `AppendEntries`, since a leader may follow a snapshot with new log entries.

2.9.3 Two-Phase Shutdown

The Phase 2 termination protocol (DONE/ACK over a per-peer REQ-REP channel) continues to run between the main threads of the replicas, but in Phase 3, it is no longer sufficient on its own: a replica must also drain its Raft-driven compaction frontier before it can safely tear down its gRPC sockets. Two changes follow.

First, between the DONE/ACK barrier and the gRPC teardown, the `Replica` blocks in `flush_snapshot(timeout_seconds)` until the Raft-committed compaction frontier (`_log_offset`) reaches the total number of operations the configuration expects every replica combined to issue — i.e. $\sum_k L[k]$ from Section 2.7. This is the point at which every operation that will ever be issued has both been folded into every stable tree and physically erased from every operation log; the `Replica`'s `tree_snapshot` now equals what its `tree` will remain for the rest of the run. The flush polls `_log_offset` on a 50ms loop and gives up after the configured deadline (default is 30s) rather than blocking indefinitely if some peer never catches up.

Second, the actual Raft teardown is split into two methods that the main thread calls in order after the flush returns:

- `close_raft_channels()` : sets the `_raft_shutdown` event, joins the `_run_raft` daemon thread (with a 2s deadline), and drops the outbound gRPC stubs held by the `GrpcTransport`. After this call the replica issues no further `RequestVoteRpc` or `AppendEntriesRpc` calls to any peer.
- `close_raft_server()` : stops the `GrpcRaftServer` with a 1s grace period. After this call the replica no longer accepts incoming Raft RPCs.

The two stages exist for a specific reason: a replica that has torn down its outbound channels but still has its inbound server running will continue answering `AppendEntries` from peers that have not yet called `close_raft_channels()`. Tearing down the outbound side first therefore avoids driving peers into unnecessary RPC timeouts while their inbound servers remain live. Conversely, stopping the inbound server before dropping outbound stubs would force every in-flight gRPC call from this replica's peers into an avoidable connection error.

2.9.4 Why `safe_index` Is the Only Command

The `safe_index` command is intentionally the only payload that ever rides on RAFT. RAFT in Phase 3 does *not* replicate the underlying CRDT operations; those still travel on the Phase 2 Publish-Subscribe `MOVE` channel, and the CRDT’s Strong Eventual Consistency does the work of converging the trees. RAFT replicates only the *agreement* that “every replica has folded its log up to safe index k ,” which is precisely the extra piece of information the local admissibility checks of Section 2.6 cannot produce on their own.

A consequence is that the RAFT log grows much more slowly than the operation log, and a single `safe_index` commit can authorise the deletion of many operation-log entries at once. This is why `__compact_log` caps the drop at `min(__raft_committed_safe_index, __safe_index_abs)`: the locally proven safe prefix might still be ahead of the RAFT-agreed prefix (no harm done; we will compact more next time), or the RAFT-agreed prefix might be ahead of the local one (which can happen on a follower that has fallen behind on its own safe-index computation, in which case the cap holds the follower back until its own checks pass).

3 The `tree-crdt` Library

This section enumerates the per-class changes between the Phase 2 and Phase 3 versions of the `tree-crdt` library. The signatures shown below are the *required* signatures; you may add helper methods and private fields as you see fit. Anything not mentioned is unchanged from Phase 2.

3.1 The `VectorClock` Class

The fields, the constructor signature, and the skeleton of the four static helpers are the same as in Phase 2 (Section 2.6.1 of the Phase 2 specification). Two contracts change:

- `update(received)` drops the self-tick on the remote path: on a remote event, take the component-wise maximum and *do not* tick (Section 2.1). The local path is unchanged.
- The four helpers implement the *total* order $\prec$ of Section 2.1 rather than the partial order of Phase 2. Concretely, `timestamp_lt` returns `True` iff `sum(lhs) < sum(rhs)`, or the sums are equal and the first left-to-right field at which they differ has `lhs[k] < rhs[k]`; `timestamp_le` composes `timestamp_lt` with `timestamp_eq`; and `timestamp_concurrent` always returns `False`, since $\prec$ admits no concurrency.

3.2 The `DeliveryClock` Class

The checkpoint vector C of Section 2.5 is held by a new concrete subclass of `Clock`, the `DeliveryClock` (`src/tree_crdt/clock/delivery.py`). Its fields and helpers mirror `VectorClock`; the one substantive difference is that `update(received)` takes an *integer source identifier* rather than a vector and increments `C[received]` by one (or the owner’s own field when `received` is `None`). Each call therefore checkpoints exactly one more operation from the named source.

3.3 The Node Class

Node reverts to the Phase 1 triple (p, m, c) : the Phase 2 producer-ID i and producer-timestamp t fields are removed (no multi-value disambiguation is needed under the single-valued semantics of Section 2.2), and the `metadata["applied"]` key is removed (no log-insertion-rollback-redo path remains). The `metadata["status"]` key and the hashing, ordering, and equality contracts carry over unchanged from Phase 2.

3.4 The Tree Class

Tree reverts to a single-valued mapping from child ID to **Node**, as in Phase 1. The method `set` is the same as Phase 2 (Section 2.6.3 of the Phase 2 specification), with the following signature simplifications driven by the loss of the multi-value `set`:

- `__call__(deleted=False) -> set[Node]` instead of `list[frozenset[Node]]`
— one node per child ID makes the outer wrapping unnecessary.
- `__getitem__(key) -> Node | None` instead of `set[Node]`.
- `get_active(key) -> Node | None`, with the orphan-detection cascade of Phase 2 collapsed to a simple parent-pointer walk.
- `move(parent, metadata, child)` loses the producer-ID and producer-timestamp arguments and the Move-Wins resolution block; concurrent Move/Delete pairs are now ordered by $\prec$ at the **Replica** layer and the later-arriving operation simply overwrites.

3.5 The Replica Class

The thread-safety strategy, the locking discipline around the clocks/trees/log, and the contracts of `apply_local_move`, `apply_remote_move`, and `current_timestamp` are the same as in Phase 2 (Section 2.6.4 of the Phase 2 specification). The remainder of this subsection lists only the deltas.

Constructor. Three new arguments, appended to the Phase 2 signature:

- `raft_base: int`: the base TCP port at which the **Replica**'s gRPC server binds; replica i listens on `host:raft_base + i`.
- `peer_hosts: list[str]`: the host of every replica indexed by replica ID, used to build the $\{\text{peer_id} \mapsto \text{host:port}\}$ table for `GrpcTransport`.
- `op_limits: dict[int, int]`: the per-replica operation-limit vector L of Section 2.7.

New fields.

- `__checkpoint_clock`: the `DeliveryClock` holding the checkpoint vector C .

- `__tree_snapshot`: the stable tree of Section 2.4. The Phase 2 tree-snapshot field is reused; the role is the same.
- `__op_log`: now a list of `(replica_id, t, old_p, old_m, p, m, c)` tuples. The `old_p` and `old_m` fields are new and record the pre-operation parent and metadata at child `c` to support exact undo.
- `__log_offset`, `__safe_index_abs`, `__last_sent_safe_index`, `__raft_committed_safe_index`: the absolute-index bookkeeping of Section 2.5, with $\text{abs} = \text{rel} + \text{log_offset}$.
- `__op_limits`: the operation-limit vector L .
- `__raft_node`, `__raft_server`, `__raft_transport`, `__raft_thread`, `__raft_shutdown`, `__pending_commits`, `__commit_lock`: the RAFT integration of Section 2.9.2. The commit lock is separate from `__lock` so the gRPC thread never blocks on user-level CRDT work.

The Phase 2 per-peer last-timestamp map and its `record_last_timestamp/get_peer_timestamp` accessors are removed: peer progress is now expressed through the checkpoint vector and RAFT commit index instead.

New methods.

- `tick_checkpoint_clock(source)`: locked wrapper around `__checkpoint_clock.update(...)`, mirroring the Phase 2 `tick_op_clock`.
- `flush_snapshot(timeout_seconds)`: blocks until `__log_offset` reaches $\sum_k L[k]$, polling at 50ms; returns `True` on success and `False` on timeout. See Section 2.9.3.
- `close_raft_channels()`, `close_raft_server()`: the two stages of the RAFT shutdown of Section 2.9.3.

New read-only properties `checkpoint_clock`, `tree_snapshot`, `raft_log`, `log_offset`, and `safe_index` expose the new state; they all return deep copies like their Phase 2 counterparts.

The `__apply_move(...)` method. The Move-Wins branches of Phase 2 are removed: under $\prec$ the method collapses to the Phase 1 shape (in-order append, or undo-do-redo around a binary-searched insertion point), and the ordering is performed according to the total order imposed by $\prec$. After the tree is consistent again, `__try_advance_safe_index()` is called once.

The `safe-index` routine. `__try_advance_safe_index()` implements the admissibility checks of Section 2.6 as follows:

1. Position itself at the relative front of the uncheckpointed log (`start_pos = __safe_index_abs - __log_offset`). Return if the suffix is empty.
2. Read the current checkpoint vector C from `__checkpoint_clock.timestamp`.

3. For every replica k that has not yet hit its `op_limits` entry, scan the suffix for an entry whose source is k and whose source-field timestamp is $C[k] + 1$. If no such entry exists, return: this corresponds to the fourth admissibility check (Section 2.6) failing for some right-of-source field k . Saturated replicas ($C[k] = L[k]$) are skipped, which is the termination story of Section 2.7.
4. Read the candidate at `start_pos`. The first three admissibility checks (single-field exceedance, source-field match, unit increment) collapse to the single test `candidate.t[k] = C[k] + 1` for the candidate's source k , because the suffix scan in step 3 already guarantees the rest. If this final check fails, return.
5. Fold the candidate into the stable tree by calling `__tree_snapshot.move(...)`, increment $C[k]$ via `__checkpoint_clock.update(k)`, and increment `__safe_index.abs` by one. Iterate.

The routine is invoked from two places: the end of `__apply_move(...)` (eagerly, every time the log changes) and the body of `__run_raft()` (on every tick, to pick up advances triggered by remote operations that landed while the RAFT thread was sleeping).

The RAFT-driven compaction routine. `__compact_log(raft_log_index)` is invoked whenever a committed `safe_index` command is drained from the pending queue. It computes the drop target `min(__raft_committed_safe_index, __safe_index.abs)`, drops the corresponding prefix from `__op_log`, raises `__log_offset` to match, and then calls `__raft_node.log.compact(...)` to truncate the RAFT log up to the same point. The `__apply_raft_command(...)` callback that feeds this routine runs on the gRPC thread and is therefore careful to enqueue rather than recurse into the replica lock.

3.6 The RaftNode, RaftLog, GrpcRaftServer, and GrpcTransport Classes

The new sub-package `src/tree_crdt/raft` is described in Sections 2.9.1–2.9.3. From the perspective of the rest of the library, the contract is:

- `RaftNode(node_id, peers, transport=None, apply=None, ...)`: constructor that the `Replica` calls with itself as the source of the apply callback. The callback receives `(command, raft_log_index)` each time a new entry is committed and applied.
- `RaftNode.set_transport(transport)`: late-binds the outbound side after the gRPC server has reported its bound port.
- `RaftNode.tick(elapsed_ms)`: the only method the `Replica`'s daemon thread calls in the steady state. Drives election and heartbeat timers, sends RPCs as needed, applies newly committed entries via the callback.
- `RaftNode.client_append(command)`: leader-only client-side append; rejected on followers. The `Replica` uses this to propose `safe_index` commands.

- `RaftNode.log.compact(index)`: trims the RAFT log up to the given absolute index, advancing the log's base index in lockstep with the `Replica's` `__log_offset`.
- `GrpcRaftServer.start()`, `GrpcRaftServer.stop(grace)`: the two endpoints of the inbound RPC lifecycle.
- `GrpcTransport.close()`: drops outbound stubs, called from `Replica.close_raft_channels()`.

The wire format is the Protobuf schema in `src/tree_crdt/raft/raft.proto`; for generating the Python files, check *README.md*.

4 Reporting Requirements

The *report.pdf* file you submit must contain, at a minimum, the following items.

1. *A worked example of the safe-index routine.* Pick a three-replica system with a non-trivial admissibility situation — ideally one in which the fourth check of Section 2.6 fires — and walk through the state of the log, the checkpoint vector, the safe-index pointer, and the stable tree at each step. Make explicit which of the four admissibility checks succeeds or fails at each candidate, and why. The example should be at least four operations long and exercise at least one tiebreak-coverage block.
2. *A termination correctness argument.* Argue, in your own words, that the safe-index routine combined with the operation-limit vector L of Section 2.7 will eventually fold every issued operation into every replica's stable tree under FIFO reliable broadcast. Address the saturated-field case explicitly: explain why skipping the fourth check on a saturated ℓ is sound, and why *not* skipping it would deadlock the run.
3. *A RAFT integration discussion.* Discuss the design choices you made (or, equivalently, that the reference implementation made on your behalf) when wiring RAFT into the `Replica`. At a minimum, address: why the only command type proposed to RAFT is `safe_index` rather than the operations themselves (Section 2.9.4); why `__compact_log(...)` caps the drop at `min(__raft_committed_safe_index, __safe_index.abs)` rather than trusting either side alone.
4. *A discussion of the conceptual delta from Phase 2.* State, explicitly, what changes to the data structure and to the conflict-resolution rules let you remove the Phase 2 multi-value semantics, the Move-Wins policy, and the "applied" metadata key. The Phase 2 specifications discussed these things; here, you should make the case in the reverse direction — given the Phase 3 total order $\prec$, why is none of that machinery still needed?

5 Package Contents

The Phase 3 starter package keeps the top-level layout of Phases 1 and 2 (`src/tree_crdt`, `tests`, `main.py`, `sample_runs`, `README.md`). The substantive differences from Phase 2 are:

- A new sub-package *src/tree_crdt/raft* containing the reference `RaftNode`, `RaftLog`, `GrpcRaftServer`, and `GrpcTransport` classes, together with the Protobuf schema in *raft.proto*.
- A new file *src/tree_crdt/clock/delivery.py* containing the `DeliveryClock` class (Section 3.2).
- Updated scaffolding for `VectorClock`, `Tree`, `Node`, and `Replica` reflecting the total order, the reversion to single-valued state, the dropped "applied" metadata key, and the new RAFT-integration fields and methods (Section 3).
- An updated *main.py* that wires in the new `RAFT_BASE` port range, passes the per-replica operation-limit vector to the `Replica` constructor, and runs the two-stage `DONE/SHUTDOWN` barrier (Section 2.9.3) followed by `flush_snapshot` and the two-stage RAFT teardown.
- An updated *pyproject.toml* that adds `grpcio-tools` to the dependency set in addition to the Phase 2 `pyzmq` and `python-dotenv` entries.
- An updated *.env*-style configuration, *.env.example*, that adds new `RAFT_BASE` and `NUM_OP_MESSAGES` entries alongside the existing `HOSTS`, `MAIN_BASE`, `LISTENER_BASE`, and `TREE_CONFIG` entries, while removing the `MAX_TIMESTAMP` entry from Phases 1 and 2.
- New sample-run outputs under *sample_runs* showing the expected behaviour with RAFT-driven log compaction. As in Phases 1 and 2, exact log content and the size of the stable tree may vary across runs without signalling a bug.
- Updated `tests` that cover the total-order helpers, the `DeliveryClock`, the safe-index admissibility checks, and the RAFT integration.

Please consult *README.md* for the up-to-date project setup instructions and the new *.env* keys.

6 Submission Guideline

The submission you make must include the following files and directories:

- The *src* directory under which you have completed your implementation of the `tree-crdt` library, including the new `DeliveryClock` class and the *src/tree_crdt/raft* sub-package.
- The *main.py* file which you have updated as the main program (with the new `RAFT_BASE` port range, the per-replica operation-limit vector, and the two-phase shutdown).
- A *report.pdf* document containing the items required by Section 4: the safe-index worked example, the termination correctness argument, the RAFT integration discussion, and the conceptual delta from Phase 2.

Place the files/directories listed above into a single ZIP archive named *CS_403-534_Project_Phase_3_Group_yourgrouppnumber.zip*, and submit it under the assignment for **Project - Phase 3** on SUCOURSE+.

Please also note that a single submission per group is sufficient.

7 Grading Criteria

- **Total Order on Vector Timestamps (8 pts):** Correct implementation of the sum-then-lexicographic order in the `VectorClock` helpers; the dropped self-tick on remote update; consistency between the update rule and the admissibility checks.
- **Single-Valued Tree and Cleanups (8 pts):** Reversion of `Tree` and `Node` to single-valued state; correct removal of the "applied" metadata key; correct cycle and orphan handling under the new shape; correct return types on `Tree.__call__` and `Tree.__getitem__`.
- **Checkpoint Vector and DeliveryClock (10 pts):** Correct `DeliveryClock` behaviour; correct exposure of the checkpoint vector to the safe-index routine.
- **Safe-Index Admissibility (20 pts):** Correct implementation of all four admissibility checks of Section 2.6, including the tiebreak coverage check; correct handling of saturated fields per Section 2.7; correct iteration and idempotence of the routine.
- **Insertion and Undo-Do-Redo Under $\prec$ (10 pts):** Correct identification of the insertion point under the total order; correct undo-do-redo sequence on out-of-order operations; correct interaction with the stable tree as the rebuild anchor.
- **RAFT Integration (20 pts):** Correct wiring of the embedded `RaftNode` (election timers, heartbeat cadence, apply callback); correct proposal of `safe_index` commands by the leader only; correct draining of committed commands without re-entering the replica lock; correct `__compact_log` bound.
- **Termination and Two-Stage Shutdown (10 pts):** Correct `flush_snapshot` polling and timeout behaviour on top of the Phase 2 `DONE/ACK` barrier; correct ordering of `close_raft_channels` and `close_raft_server`.
- **Robustness, Thread-Safety, Runtime Orchestration (8 pts):** Reliable behaviour under edge cases and malformed operations; preserved tree, log, and checkpoint-vector invariants; thread-safe concurrent access between the listener thread, the main thread, the RAFT thread, and the gRPC server thread; graceful shutdown without leaked gRPC servers.
- **Report Quality (6 pts):** Clarity and completeness of the four items required by Section 4.